

DevOps

Note: This page summarizes James Shore’s articles on DevOps and Build for Operation from *The Art of Agile Development, Second Edition*.

Breaking Down Walls Between Development, Operations, and Security

DevOps is close collaboration between development, operations, and security. By including people with these skills on the same team—or at least involving them in decisions—you build operability and security into software from the start, rather than adding them as an afterthought.

That’s really all there is to it. The fundamental idea is simple. The hard part is actually doing it.

Why DevOps Matters

In traditional organizations, developers build software and hand it off for release. Operations is a distant department. Security reviews happen at the end. This creates walls—and walls create problems.

When developers don’t understand production needs, they build software that’s hard to monitor and manage. When operations can’t influence design decisions, they’re stuck firefighting problems that could have been prevented. When security is an afterthought, vulnerabilities get baked in.

DevOps breaks these walls down. Cross-functional teams include (or regularly collaborate with) people who understand deployment, monitoring, and security. This allows you to create software that’s safer, more reliable, and easier to manage.

Building for Operation

Building for operation means “shifting left”—thinking about security and operations needs from the beginning, not the end. Include ops and security people in planning sessions. Create stories for making software easier to monitor, manage, and secure. Prioritize those stories alongside feature work.

Don’t save operations and security stories for the end of development. When you add a feature that requires a new database, also add stories for provisioning, securing, monitoring, backing up, and restoring that database.

Key areas to consider:

Threat Modeling — Diagram your system architecture, brainstorm what can go wrong, and decide what to do about it. This security technique helps you understand operational needs too.

Configuration — Separate environment configuration (database strings, API keys) from code. Deploy the same code everywhere; inject different configuration per environment.

Secrets — Passwords and API keys need special handling. Most team members shouldn’t have access. Define secure procedures for generating, storing, rotating, and auditing secrets. Never write them to logs.

Logging and Monitoring — Think about what can go wrong and what people will need to diagnose problems at 3am. Use structured logs routed to a centralized store. Accompany alerts with documentation explaining what they mean and what to do.

Observability — Logs and metrics together create observability: the ability to understand system behavior from technical, business, security, and support perspectives.

Continuous Integration and Deployment

Continuous Integration (CI) means integrating code changes frequently—at least daily, often multiple times per day. Each integration triggers an automated build and test run. This catches integration problems early, when they’re cheap to fix.

Continuous Deployment extends this: every change that passes tests gets deployed to production automatically. This sounds scary, but it reduces risk. Small, frequent deployments are easier to troubleshoot than large, infrequent ones. When something goes wrong, there’s less to investigate.

The prerequisites are solid: good test coverage, feature flags to deploy incomplete work safely, and the ability to roll back quickly. Teams that achieve continuous deployment often deploy dozens of times per day with little drama.

Avoiding the “DevOps Team” Cargo Cult

A common misunderstanding is creating a separate “DevOps team” that sits between development and operations. This defeats the purpose. You’ve just added another silo with another handoff.

True DevOps means embedding operations and security skills within development teams—or at minimum, establishing close ongoing collaboration. When developers feel production pain directly (through on-call rotations, for example), they write code that makes better alerting decisions and is easier to operate.

If you don’t have ops or security people on your team, reach out to those departments early. Let them know you want their input on operational requirements before anything catches fire. They’re usually eager to help.

Making Time for Operations

“We don’t have time for this” is a common objection. But you don’t have time *not* to. Teams working on software that isn’t built for operation typically waste enormous time firefighting problems that better observability would have prevented or quickly diagnosed.

Treat alerts like bugs: they should be unexpected and taken seriously. Every alert should be addressed—either by fixing the underlying issue or improving the alert so it stops crying wolf.

References

- James Shore - DevOps (Introduction) — Overview of DevOps in *The Art of Agile Development, Second Edition*.
- James Shore - Build for Operation — Detailed guidance on security, configuration, logging, and monitoring.
- Gene Kim et al., *The DevOps Handbook* (2016) — Comprehensive guide to DevOps principles and practices.
- Gene Kim et al., *The Phoenix Project* (2013) — A novel about introducing DevOps to an organization.